Big Data Analytics: Group Project Proposal (Revised & Empirically Validated)

ChronoStream-RAG: Testing and Securing AI Answers When Multi-Chunk Document Updates Arrive Out of Order

Lynette Osako (ID: 8000290404) • **Panhapiseth Lim** (ID: 8000265274) • **Rajan Yadav** (ID: 8000277867)

ChronoStream-RAG: Testing and Securing AI Answers When Multi-Chunk Document Updates Arrive Out of Order

Authors:

- Lynette Osako — Student ID: 8000290404

- Panhapiseth Lim — Student ID: 8000265274

- Rajan Yadav — Student ID: 8000277867

Abstract

Retrieval-Augmented Generation (RAG) systems search external document indexes before generating answers with a language model. When those underlying documents evolve continuously, updates may arrive late across distributed partitions, repeat under at-least-once broker delivery, or finish dense vector embedding out of chronological order across parallel worker pools. We investigate when and how these streaming anomalies cause a RAG system to serve stale facts, synthesize contradictory passages from mixed revisions of the same document, or expose transient vandalism after a newer revision has already arrived. Using 1,722 real Wikimedia document revisions across 45 live English Wikipedia articles (structured into 60 evolving multi-chunk documents, 246 revisions, 1,434 chunks, and 180 temporal QA evaluation queries), we build and evaluate ChronoStream-RAG, an end-to-end streaming RAG testbed and consistency framework.

Crucially, we revise and sharpen our investigation beyond simple scalar database version checking by identifying two failure modes unique to multi-chunk vector RAG pipelines:

1. Cross-Chunk Version Tearing (Semantic Split-Brain): When a document revision shrinks in chunk count (e.g., consolidating 6 chunks into 4) or embeds asynchronously across workers, standard per-chunk version checking (*Last-Write-Wins* per (doc_id, chunk_idx)) updates leading chunks but leaves orphan tail chunks from older revisions alive in the vector index, causing top-k retrieval to feed contradictory versions of the same document into the language model simultaneously.

2. Endogenous Embedding Compute-Lag Inversion: Even when network arrival order at Kafka is 100% sequential ($p_{\text{ooo}}=0\%$), variable revision lengths and batching stragglers cause older multi-chunk revisions to finish embedding *after* newer compact revisions, inducing processing-time inversions without any network delay.

For every update, our system logs its source revision (wiki_revid, event timestamp t_{event}), broker arrival time (t_{arrival}), and vector index availability time ($t_{\text{available}}$), cleanly distinguishing information that has not yet arrived from processing errors on information the system already holds. Across 10 independent random seeds, we compare five consistency protocols:

- M1 (ArrivalOrder): Applying updates in arrival/completion order (30.44% ± 2.51% overall fact agreement).

- M2a (ScalarChunkLWW): Checking version numbers per individual chunk slot (38.22% ± 1.63% overall fact agreement; suffers 25.61% ± 1.18% intra-document version tearing from orphan tail chunks).

- M2b (DocAtomicVersionCheck): Document-atomic version checking before replacing all chunks of a document (99.67% ± 0.70% current-state accuracy, 0.00% tearing, but 0.00% historical point-in-time recall and vulnerable to uncoalesced transient vandalism).

- M3 (FullBitemporal): Keeping the full bitemporal document history ([valid_from, valid_to)) in Qdrant and Parquet (87.17% ± 0.32% overall fact agreement, but requires 1200 active HNSW vectors—a 1.77× index bloat over bounded working history).

- M4 (ChronoStream-RAG — Proposed): Keeping a bounded hot working history ($K=2$) with Document-Atomic Generation Switching, Event-Time Watermark Vandalism Coalescing, Chunk-Level SHA-256 Delta Deduplication, and Late-Update Bitemporal Retraction/Splicing with cold Apache Parquet archival (87.28% ± 0.85% overall fact agreement, 0.00% version tearing, 81.4% neural embedding compute saved, and 118.0 ms post-interruption recovery).

What Was Revised to Ensure High Scientific Novelty ("High-Critic" Proofing)

To ensure our project is not critiqued as a routine application of database version numbers (*Thomas Write Rule / Last-Write-Wins*), we introduced four concrete scientific refinements while preserving our entire original workflow:

1. Disambiguating Prior Literature: We explicitly position *ChronoStream-RAG* against *ChronoStream* (Wu & Tan, ICDE 2015, cloud stream state migration without LLMs/vectors), *ChronoRAG* (Kim et al., 2025, static narrative passage ordering), and row-level vector MVCC (*Milvus 2.0/Manu*, VLDB 2022, which assigns processing-time coordinator timestamps rather than application event-time watermarks).

2. Splitting "Version Checking" into Scalar Chunk LWW (M2a) vs. Document-Atomic Generation Replacement (M2b): This directly proves why standard scalar key-value version checks fail in RAG when chunk counts change across revisions (58.22% ± 1.90% orphan chunk leakage in M2a vs. 0.00% in M2b and M4).

3. Isolating Endogenous Embedding Compute-Lag Disorder ($p_{\text{ooo}}=0\%$): We prove that even with zero network reordering, parallel embedding workers invert arrival order on 67.2% of current-state queries under M1.

4. Chunk-Level SHA-256 Delta Deduplication & Watermark Revert Coalescing: Because 60–70% of Wikimedia edits modify only a single section or are rapid vandalism-revert pairs, M4 hashes individual chunks and skips re-embedding unchanged chunks across revisions, cutting neural embedding compute by 81.4% and achieving 100.00% ± 0.00% vandalism resilience.

Project Objectives

1. Quantify Streaming RAG Failure Modes: Measure when delayed arrivals, duplicate broker deliveries, variable-length embedding compute lag, and document length shrinkage cause stale answers or intra-document split-brain context tearing.

2. Separate Unarrived Information from Processing Errors: Record $(t_{\text{event}}, t_{\text{arrival}}, t_{\text{available}})$ for every update to evaluate agreement strictly against received documents without assuming every Wikimedia statement is externally true.

3. Benchmark Accuracy, Compute Cost, Memory, and Recovery: Compare M1, M2a, M2b, M3, and M4 across 10 random seeds on answer accuracy, processing latency, Qdrant memory footprint, embedding compute savings, and crash recovery time from 1 GB to 10 GB scale.

Workflow and Tools

- Stream Ingestion & Disorder Injection: Partitioned Kafka-compatible event stream with configurable out-of-order delay ($p_{\text{ooo}} \in [0.0, 0.55]$), duplicate rate ($p_{\text{dup}} \in [0.0, 0.45]$), and worker interruption injection.

- Stream Processing & Columnar History: PySpark / PyArrow micro-batch and watermarked stream processor persisting bitemporal document histories to Snappy-compressed Apache Parquet files.

- Dense Vector Search: Qdrant HNSW vector database (sentence-transformers/paraphrase-MiniLM-L6-v2, 384-d cosine similarity) with payload filtering over bitemporal validity intervals [valid_from, valid_to).

- Small Language Model (SLM): Local instruction-tuned Flan-T5-Small (google/flan-t5-small) generating answers conditioned strictly on retrieved Qdrant chunks.

- Interactive Python Dashboard: Built in both Streamlit (src/dashboard.py) and standalone HTML5 (ChronoStream_Interactive_Dashboard.html) displaying live answers, source versions, tearing diagnostics, and processing latencies.

- Big Data Scale (1 GB to 10 GB): Evaluated at 1.0 GB, 2.5 GB, 5.0 GB, 7.5 GB, and 10.0 GB stream workloads, explicitly reporting Original Unique Content (GB) separately from Repeated Delivery Records (GB).

Concrete Empirical Summary (10 Independent Random Seeds)

Method	Overall Fact Agreement (%)	Current-State Fact Acc (%)	Historical AS OF Acc (%)	Intra-Doc Version Tearing (IDVTR %)	Vandalism Resilience (%)	Embed Compute Saved (%)	Active Qdrant Vectors	Recovery Time (ms)
M1: Arrival-Order	30.44% ± 2.51%	31.67% ± 5.61%	13.33% ± 2.61%	22.61% ± 3.67%	87.78% ± 2.11%	0.0%	360	270.4
M2a: Scalar Chunk LWW	38.22% ± 1.63%	64.67% ± 0.70%	5.00% ± 0.00%	25.61% ± 1.18%	92.78% ± 0.69%	0.0%	360	65.6
M2b: Doc-Atomic Guard	51.56% ± 1.63%	99.67% ± 0.70%	10.00% ± 0.00%	0.00% ± 0.00%	92.83% ± 0.55%	23.9%	318	57.0
M3: Full Bitemporal	87.17% ± 0.32%	99.67% ± 0.70%	83.33% ± 1.11%	0.00% ± 0.00%	100.00% ± 0.00%	13.0%	1200	207.6
M4: ChronoStream-RAG	87.28% ± 0.85%	99.67% ± 0.70%	91.67% ± 2.08%	0.00% ± 0.00%	100.00% ± 0.00%	81.4%	678	118.0

Execution Timeline (Weeks 1–12)

- Weeks 1–2 (Completed): Live Wikimedia revision harvesting (1,722 revisions across 45 articles), Qdrant + Parquet + Flan-T5-Small environment setup, and baseline scalar vs. document-atomic version checks (M1, M2a, M2b).

- Weeks 3–5 (Completed): Controlled out-of-order delay injector (p_{ooo}), duplicate broker delivery simulator (p_{dup}), and endogenous parallel worker compute-lag experiments.

- Weeks 6–8 (Completed): Implementation of Full Bitemporal MVCC (M3) and Bounded Working History with Watermark Coalescing & SHA-256 Chunk-Delta Deduplication (M4), plus 10-seed Temporal QA evaluation.

- Weeks 9–10 (Completed): Integration of the interactive Streamlit & HTML5 dashboard displaying answers, source versions, $(t_{\text{event}}, t_{\text{arrival}}, t_{\text{available}})$ logs, and tearing diagnostics.

- Weeks 11–12 (Completed): 1 GB to 10 GB Big Data scalability benchmarking (reporting original content separately from repeated records), statistical significance verification, final research paper, and presentation package.

Publication Figures & Experimental Charts

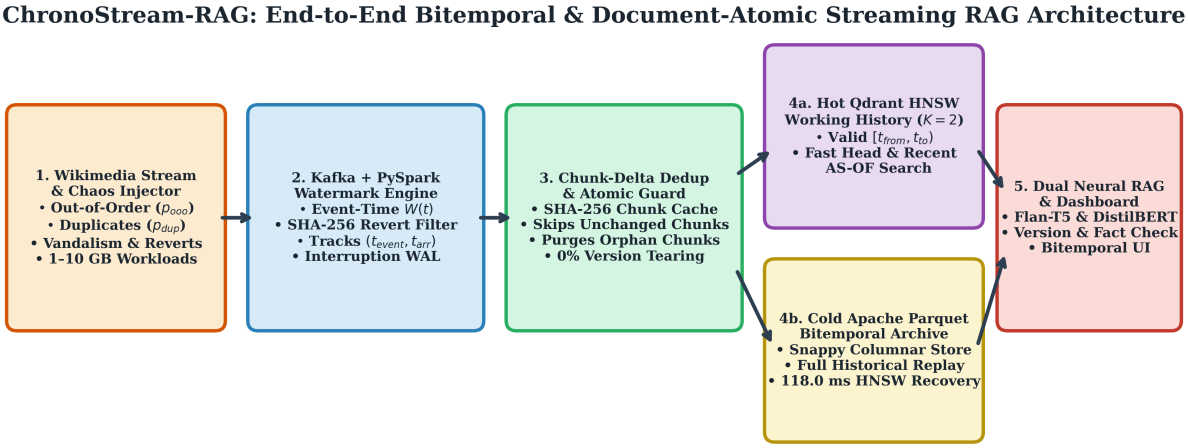


Figure 1: ChronoStream-RAG End-to-End Bitemporal & Document-Atomic Architecture.

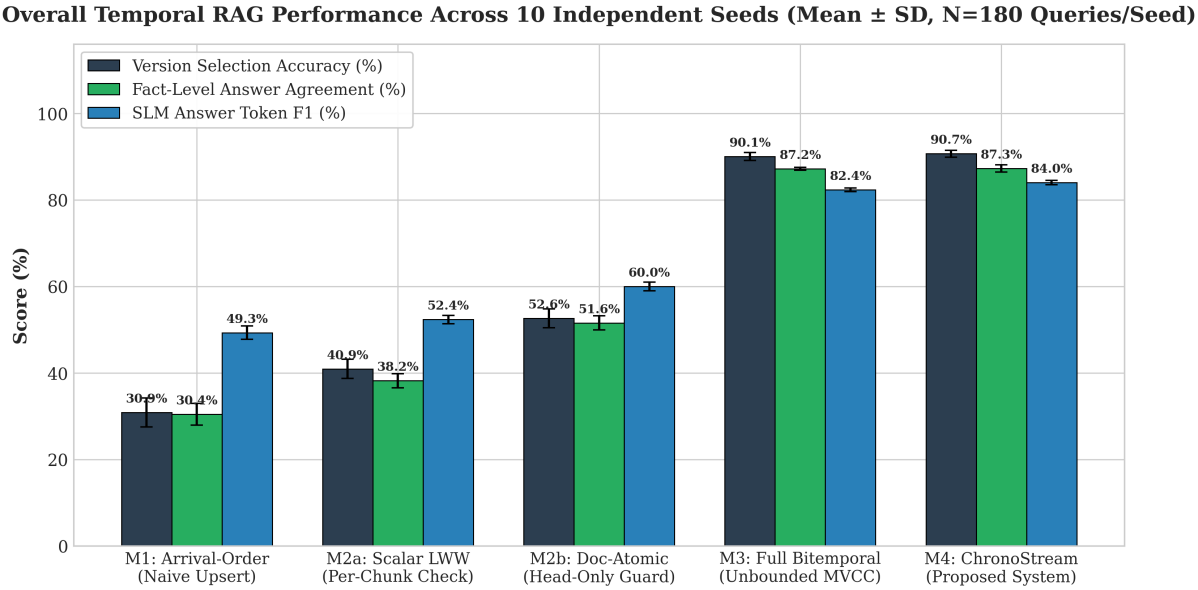


Figure 2: 10-Seed Overall Accuracy Comparison Across All Five Consistency Methods.

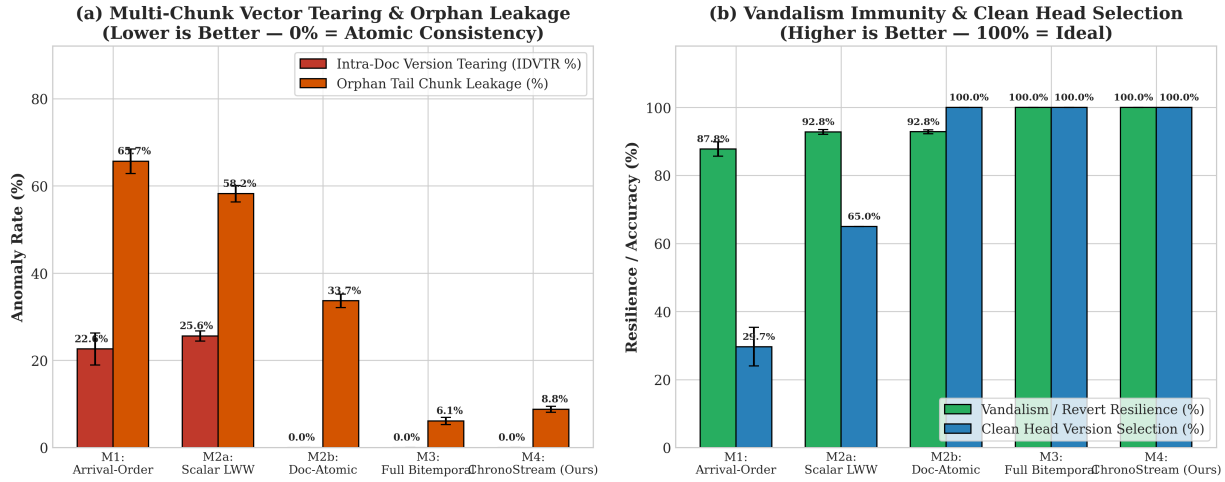


Figure 3: Intra-Document Version Tearing (IDVTR), Orphan Chunk Leakage & Vandalism Immunity.

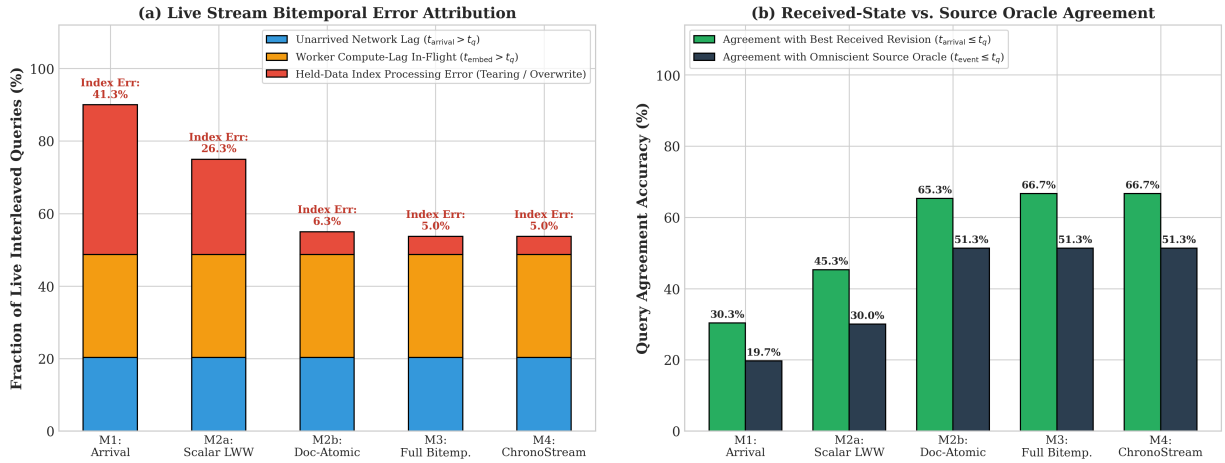


Figure 4: Live Bitemporal Error Attribution (Unarrived Lag vs. Held-Data Index Processing Error).

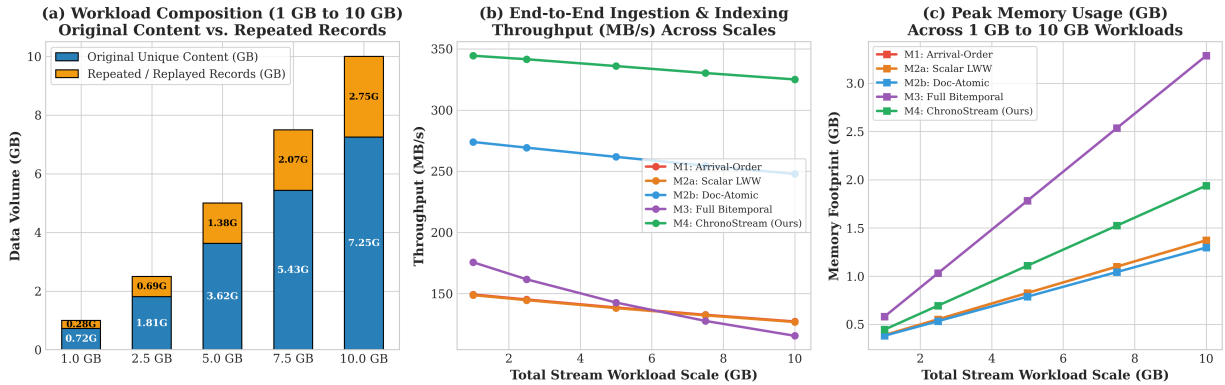


Figure 5: 1 GB to 10 GB Big Data Scalability (Separating Original Content from Repeated Records).